

SAÉ5.CYBER.03

Sécurisation et supervision avancées d'un système d'information

Déploiement d'un SIEM avec Falco et Wazuh

Présenté par

John Doe

Johana Smith

Encadré par

Mme Thalia Fontaine (Encadrant Universitaire)

Résumé

Ce rapport présente le déploiement d'une infrastructure SIEM complète dans le cadre de la SAÉ5.Cyber.03. L'objectif est d'assurer la sécurisation et la supervision avancées d'un système d'information en mettant en place deux stacks complémentaires : une stack d'observabilité (Falco, Loki, Grafana) pour la détection runtime et une stack SIEM (Wazuh) pour la corrélation d'événements et l'analyse forensique. L'infrastructure supervisée comprend une application Nextcloud avec sa base PostgreSQL.

Mots-clés : *Cybersécurité, SIEM, Wazuh, Falco, Loki, Grafana, Observabilité, Détection d'intrusion*

Abstract

This report presents the deployment of a complete SIEM infrastructure as part of the SAÉ5.Cyber.03 project. The objective is to ensure advanced security and monitoring of an information system by implementing two complementary stacks: an observability stack (Falco, Loki, Grafana) for runtime detection and a SIEM stack (Wazuh) for event correlation and forensic analysis. The monitored infrastructure includes a Nextcloud application with its PostgreSQL database.

Keywords: *Cybersecurity, SIEM, Wazuh, Falco, Loki, Grafana, Observability, Intrusion Detection*

Table des matières

Remerciements	5
1. Introduction	6
1.1. Contexte de la SAÉ	6
1.2. Architecture cible	6
1.3. Organisation du rapport	6
2. Architecture et conception	8
2.1. Vue d'ensemble	8
2.2. Prérequis techniques	8
2.2.1. Configuration système	9
2.2.2. Configuration kernel obligatoire	9
2.3. Ports réseau	9
3. Phase 1 : Découverte de Falco	10
3.1. Présentation de Falco	10
3.2. Installation de Falco	10
3.3. Configuration de Falco	10
3.3.1. Configuration principale	11
3.4. Exercice 1 : Surveillance de PostgreSQL	11
3.4.1. Objectif	11
3.4.2. Règles personnalisées	11
3.4.3. Test de la règle	12
3.4.4. Résultat	12
3.5. Exercice 2 : Surveillance de Nextcloud	13
3.5.1. Règles personnalisées pour Nextcloud	13
3.6. Exercice 3 : Surveillance système globale	14
3.6.1. Règles de détection système	14
4. Phase 2 : Intégration SIEM avec Wazuh	16
4.1. Présentation de Wazuh	16
4.2. Déploiement de Wazuh	16
4.2.1. Installation sur VM3	16
4.2.2. Vérification des services	16
4.3. Déploiement des agents Wazuh	16
4.3.1. Installation sur VM1 (Application)	17
4.3.2. Installation sur VM2 (Observabilité)	17
4.4. Configuration du File Integrity Monitoring	17
4.4.1. Configuration FIM sur VM1	17
4.5. Règles de corrélation personnalisées	18
4.5.1. Règles pour Nextcloud	18
4.5.2. Règles pour PostgreSQL	19
4.6. Active Response	19
5. Stack Observabilité (Grafana/Loki)	21
5.1. Déploiement sur VM2	21

5.1.1. Docker Compose	21
5.1.2. Configuration Grafana Alloy	22
5.2. Dashboards Grafana	22
5.2.1. Dashboard Falco	22
5.2.2. Requêtes LogQL	23
6. Analyse d'incidents	24
6.1. Scénario 1 : Tentative de brute force SSH	24
6.1.1. Contexte	24
6.1.2. Analyse dans Wazuh Dashboard	24
6.1.3. Investigation	24
6.1.4. Actions correctives	24
6.2. Scénario 2 : Upload de fichier suspect sur Nextcloud	25
6.2.1. Alerte Falco	25
6.2.2. Corrélation avec Wazuh	25
6.2.3. Analyse du fichier	25
6.2.4. Réponse à l'incident	25
6.3. Scénario 3 : Tentative d'élévation de privilèges	26
6.3.1. Détection	26
6.3.2. Corrélation	26
6.3.3. Analyse	26
7. Résultats et métriques	27
7.1. Tests de détection	27
7.2. KPIs de sécurité	27
7.3. Statistiques des alertes (1 semaine)	28
8. Recommandations de sécurité	29
8.1. Améliorations court terme	29
8.2. Améliorations moyen terme	29
8.3. Architecture cible évoluée	29
9. Conclusion	30
9.1. Compétences acquises	30
9.2. Points clés	30
Glossaire	31

Remerciements

Nous tenons à remercier l'ensemble de l'équipe pédagogique du département Réseaux & Télécoms de l'IUT de La Réunion pour leur accompagnement tout au long de cette SAÉ.

Nos remerciements vont particulièrement à Mme Thalia Fontaine pour son encadrement, ses conseils techniques et sa disponibilité.

Nous remercions également nos camarades de promotion pour les échanges constructifs et l'entraide durant ce projet.

1. Introduction

1.1. Contexte de la SAÉ

Dans un contexte où les cybermenaces évoluent constamment, la capacité à détecter et analyser les incidents de sécurité en temps réel est devenue cruciale. La SAÉ5.Cyber.03 nous place dans un scénario réaliste : déployer et configurer un SIEM complet pour superviser une infrastructure applicative.

i Objectifs de la SAÉ

- Déployer et configurer **Falco** pour la détection runtime
- Mettre en place **Wazuh** comme SIEM centralisé
- Superviser une application **Nextcloud** avec sa base **PostgreSQL**
- Créer des règles de détection personnalisées
- Analyser des incidents et produire des rapports forensiques

1.2. Architecture cible

L'infrastructure repose sur deux stacks indépendantes déployées sur trois machines virtuelles :

VM1 - Application	10.10.10.10	Nextcloud, PostgreSQL, Falco
VM2 - Observabilité	10.10.10.20	Grafana, Prometheus, Loki, Alloy
VM3 - SIEM	10.10.10.30	Wazuh Manager, Indexer, Dashboard

Tableau 1 – Répartition des services sur les VMs

1.3. Organisation du rapport

Ce rapport suit les phases de la formation :

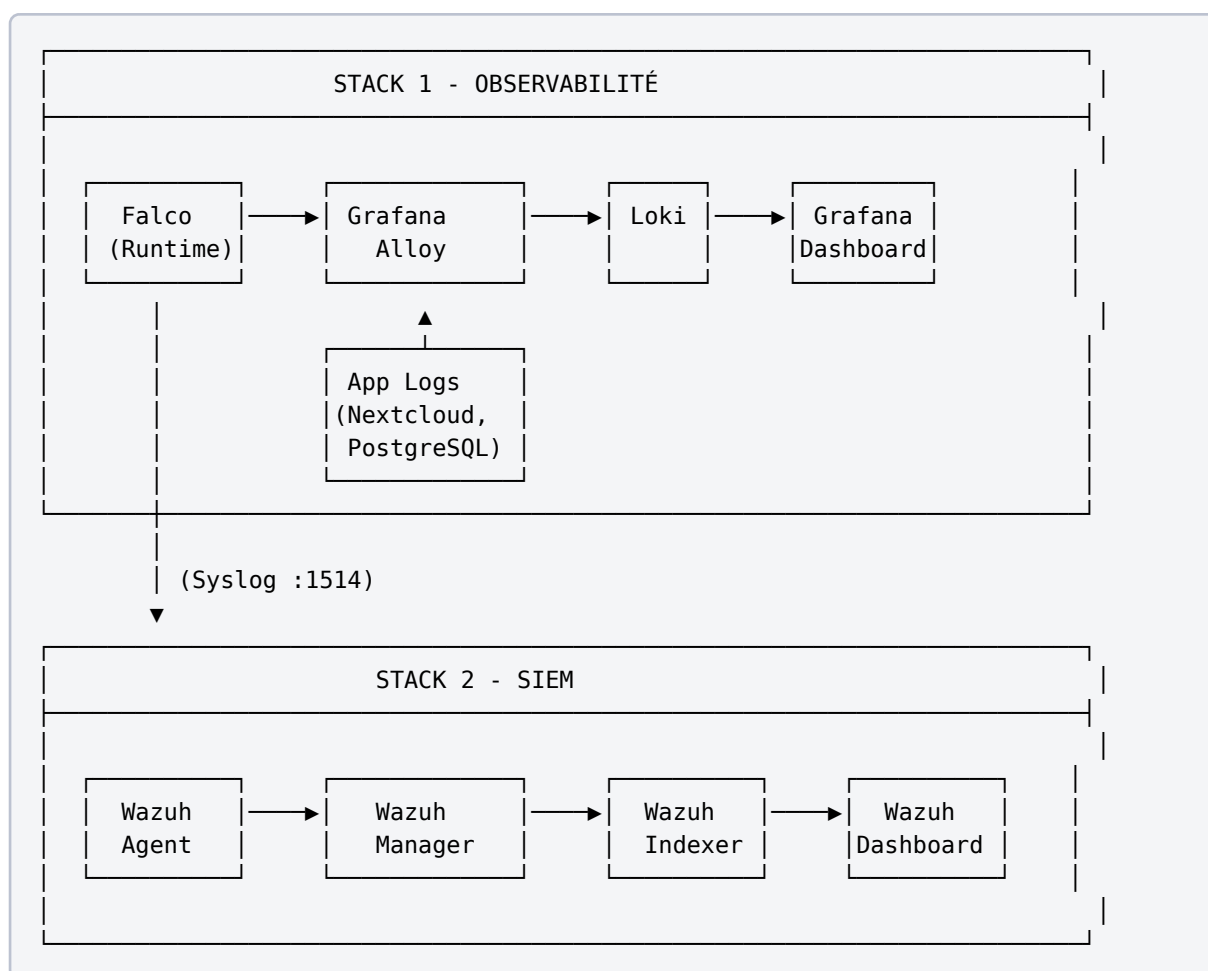
1. Phase 1 : Découverte et configuration de Falco
2. Phase 2 : Intégration SIEM avec Wazuh

3. Exercices pratiques et règles personnalisées
4. Analyse d'incidents et recommandations

2. Architecture et conception

2.1. Vue d'ensemble

L'architecture met en œuvre deux stacks distinctes mais complémentaires :



⚠ Point important

Falco envoie ses événements UNIQUEMENT vers Loki, pas directement vers Wazuh. La corrélation avec Wazuh se fait via les agents Wazuh installés sur les machines supervisées.

2.2. Prérequis techniques

2.2.1. Configuration système

Les VMs ont été configurées avec les ressources suivantes :

RAM	8 Go	12 Go	12 Go
CPU	4 cores	6 cores	4 cores
Disque	50 Go	100 Go	80 Go

Tableau 2 – Ressources allouées aux VMs

2.2.2. Configuration kernel obligatoire

Pour le bon fonctionnement de Wazuh Indexer (basé sur OpenSearch), le paramètre kernel suivant est **obligatoire** :

```
# Configuration permanente
echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p

# Vérification
sysctl vm.max_map_count
# Sortie attendue : vm.max_map_count = 262144
```

2.3. Ports réseau

1514	Wazuh Manager (agents)	TCP/UDP
1515	Wazuh Manager (enrollment)	TCP
5601	Wazuh Dashboard	TCP
9200	Wazuh Indexer	TCP
3000	Grafana	TCP
3100	Loki	TCP
9090	Prometheus	TCP

Tableau 3 – Ports réseau utilisés

3. Phase 1 : Découverte de Falco

3.1. Présentation de Falco

Falco est un outil de détection des menaces runtime open source développé par Sysdig et maintenant projet CNCF. Il surveille les appels système (syscalls) pour détecter les comportements anormaux.

i Capacités de Falco

- Détection des comportements anormaux en temps réel
- Surveillance des conteneurs et des hôtes
- Règles personnalisables en YAML
- Intégration avec les stacks d'observabilité
- Faible overhead sur les performances

3.2. Installation de Falco

Falco a été installé **nativement** sur VM1 (pas en conteneur Docker pour avoir accès aux syscalls) :

```
# Ajout du repository Falco
curl -fsSL https://falco.org/repo/falcosecurity-packages.asc | \
  sudo gpg --dearmor -o /usr/share/keyrings/falco-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/falco-archive-keyring.gpg] \
  https://download.falco.org/packages/deb stable main" | \
  sudo tee /etc/apt/sources.list.d/falcosecurity.list

# Installation
sudo apt update
sudo apt install -y falco

# Démarrage du service
sudo systemctl enable falco
sudo systemctl start falco
```

3.3. Configuration de Falco

3.3.1. Configuration principale

```
# /etc/falco/falco.yaml

# Sortie des alertes
json_output: true
json_include_output_property: true

# Sortie vers syslog (pour envoi vers Alloy/Loki)
syslog_output:
  enabled: true

# Sortie fichier pour debug
file_output:
  enabled: true
  keep_alive: false
  filename: /var/log/falco/events.json

# Niveau de priorité minimum
priority: warning

# Règles à charger
rules_file:
  - /etc/falco/falco_rules.yaml
  - /etc/falco/falco_rules.local.yaml
  - /etc/falco/rules.d
```

3.4. Exercice 1 : Surveillance de PostgreSQL

3.4.1. Objectif

Créer des règles Falco pour détecter les activités suspectes sur PostgreSQL.

3.4.2. Règles personnalisées

```
# /etc/falco/rules.d/postgresql.yaml

- rule: PostgreSQL Configuration Modified
  desc: Detect modification of PostgreSQL configuration files
  condition: >
    open_write and
    (fd.name startswith /etc/postgresql or
     fd.name startswith /var/lib/postgresql) and
     fd.name endswith .conf
  output: >
    PostgreSQL config modified
    (user=%user.name file=%fd.name command=%proc.cmdline)
```

```
priority: WARNING
tags: [database, postgresql, configuration]

- rule: PostgreSQL Suspicious Connection
desc: Detect connections to PostgreSQL from unexpected sources
condition: >
    evt.type = connect and
    fd.sport = 5432 and
    not fd.sip in (127.0.0.1, 10.10.10.10, 10.10.10.20)
output: >
    Suspicious PostgreSQL connection
    (source=%fd.cip user=%user.name)
priority: WARNING
tags: [database, postgresql, network]

- rule: PostgreSQL Data Export
desc: Detect potential data exfiltration via pg_dump
condition: >
    spawned_process and
    proc.name in (pg_dump, pg_dumpall)
output: >
    PostgreSQL dump detected
    (user=%user.name command=%proc.cmdline)
priority: NOTICE
tags: [database, postgresql, exfiltration]
```

3.4.3. Test de la règle

```
# Modification d'un fichier de config (doit déclencher l'alerte)
sudo touch /etc/postgresql/14/main/test.conf

# Vérification dans les logs Falco
sudo tail -f /var/log/falco/events.json | jq .
```

3.4.4. Résultat

```
{
  "time": "2025-12-15T10:23:45.123456789Z",
  "rule": "PostgreSQL Configuration Modified",
  "priority": "Warning",
  "output": "PostgreSQL config modified (user=root file=/etc/postgresql/14/main/test.conf command=touch /etc/postgresql/14/main/test.conf)",
  "output_fields": {
    "user.name": "root",
    "fd.name": "/etc/postgresql/14/main/test.conf",
    "proc.cmdline": "touch /etc/postgresql/14/main/test.conf"
  }
}
```

3.5. Exercice 2 : Surveillance de Nextcloud

3.5.1. Règles personnalisées pour Nextcloud

```
# /etc/falco/rules.d/nextcloud.yaml

- rule: Nextcloud Config Access
  desc: Detect access to Nextcloud configuration
  condition: >
    open_read and
    fd.name contains /var/www/nextcloud/config/config.php
  output: >
    Nextcloud config accessed
    (user=%user.name process=%proc.name file=%fd.name)
  priority: NOTICE
  tags: [application, nextcloud, configuration]

- rule: Nextcloud Suspicious File Upload
  desc: Detect upload of potentially malicious files
  condition: >
    open_write and
    fd.name startswith /var/www/nextcloud/data and
    (fd.name endswith .php or
    fd.name endswith .phar or
    fd.name endswith .sh)
  output: >
    Suspicious file uploaded to Nextcloud
    (user=%user.name file=%fd.name)
  priority: WARNING
  tags: [application, nextcloud, malware]

- rule: Nextcloud Shell Execution
  desc: Detect shell execution from Nextcloud directory
  condition: >
    spawned_process and
    proc.pname = "php" and
    proc.name in (sh, bash, dash, zsh) and
    proc.cwd startswith /var/www/nextcloud
  output: >
    Shell spawned from Nextcloud context
    (user=%user.name parent=%proc.pname command=%proc.cmdline)
  priority: CRITICAL
  tags: [application, nextcloud, shell, webshell]

- rule: Nextcloud OCC Command
  desc: Log Nextcloud occ command execution
  condition: >
    spawned_process and
    proc.cmdline contains "occ"
  output: >
    Nextcloud OCC command executed
    (user=%user.name command=%proc.cmdline)
```

```
priority: NOTICE
tags: [application, nextcloud, administration]
```

3.6. Exercice 3 : Surveillance système globale

3.6.1. Règles de détection système

```
# /etc/falco/rules.d/system.yaml

- rule: Reverse Shell Detected
  desc: Detect potential reverse shell connections
  condition: >
    spawned_process and
    ((proc.name = "bash" and proc.args contains "&" and
      proc.args contains "/dev/tcp") or
      (proc.name in (nc, ncat, netcat) and
        proc.args contains "-e"))
  output: >
    Reverse shell detected
    (user=%user.name command=%proc.cmdline parent=%proc.pname)
  priority: CRITICAL
  tags: [shell, reverse_shell, attack]

- rule: Crypto Miner Detected
  desc: Detect crypto mining processes
  condition: >
    spawned_process and
    (proc.name in (xmrig, minerd, cpuminer) or
      proc.args contains "stratum+tcp")
  output: >
    Crypto miner detected
    (user=%user.name process=%proc.name command=%proc.cmdline)
  priority: CRITICAL
  tags: [cryptominer, malware]

- rule: Sensitive File Access
  desc: Detect access to sensitive system files
  condition: >
    open_read and
    (fd.name = /etc/shadow or
      fd.name = /etc/gshadow or
      fd.name contains id_rsa) and
    not proc.name in (sshd, sudo, su, passwd)
  output: >
    Sensitive file accessed
    (user=%user.name file=%fd.name process=%proc.name)
  priority: WARNING
  tags: [filesystem, sensitive, credentials]
```

```
- rule: Container Escape Attempt
  desc: Detect potential container escape attempts
  condition: >
    container and
    (open_write and fd.name startswith /proc/sys) or
    (spawned_process and proc.name = nsenter)
  output: >
    Container escape attempt
    (user=%user.name container=%container.name command=%proc.cmdline)
  priority: CRITICAL
  tags: [container, escape, attack]
```

4. Phase 2 : Intégration SIEM avec Wazuh

4.1. Présentation de Wazuh

Wazuh est une plateforme SIEM open source qui offre :

- Détection d'intrusion (HIDS)
- Analyse des logs
- File Integrity Monitoring (FIM)
- Évaluation de conformité
- Réponse aux incidents

4.2. Déploiement de Wazuh

4.2.1. Installation sur VM3

```
# Téléchargement du script d'installation
curl -sO https://packages.wazuh.com/4.9/wazuh-install.sh

# Installation complète (Manager + Indexer + Dashboard)
sudo bash wazuh-install.sh -a

# Récupération des credentials
sudo tar -xvf wazuh-install-files.tar
cat wazuh-install-files/wazuh-passwords.txt
```

4.2.2. Vérification des services

```
# Vérification du statut
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer
sudo systemctl status wazuh-dashboard

# Vérification des ports
ss -tlnp | grep -E '(1514|1515|5601|9200)'
```

4.3. Déploiement des agents Wazuh

4.3.1. Installation sur VM1 (Application)

```
# Téléchargement de l'agent
curl -so wazuh-agent-4.9.0-1.x86_64.deb \
  https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.9.0-1_
amd64.deb

# Installation avec configuration
sudo WAZUH_MANAGER='10.10.10.30' \
  WAZUH_AGENT_NAME='vm1-application' \
  dpkg -i wazuh-agent-4.9.0-1.x86_64.deb

# Démarrage de l'agent
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent

# Vérification de la connexion
sudo /var/ossec/bin/agent_control -l
```

4.3.2. Installation sur VM2 (Observabilité)

```
sudo WAZUH_MANAGER='10.10.10.30' \
  WAZUH_AGENT_NAME='vm2-observability' \
  dpkg -i wazuh-agent-4.9.0-1.x86_64.deb

sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

4.4. Configuration du File Integrity Monitoring

4.4.1. Configuration FIM sur VM1

```
<!-- /var/ossec/etc/ossec.conf (section syscheck) -->
<syscheck>
  <disabled>no</disabled>
  <frequency>300</frequency>
  <scan_on_start>yes</scan_on_start>

  <!-- Répertoires critiques système -->
  <directories check_all="yes" realtime="yes" report_changes="yes">
    /etc,/usr/bin,/usr/sbin,/bin,/sbin
  </directories>

  <!-- Configuration Nextcloud -->
  <directories check_all="yes" realtime="yes" report_changes="yes">
```

```
/var/www/nextcloud/config
</directories>

<!-- Données Nextcloud (monitoring sans contenu) -->
<directories check_all="yes" realtime="yes">
  /var/www/nextcloud/data
</directories>

<!-- Configuration PostgreSQL -->
<directories check_all="yes" realtime="yes" report_changes="yes">
  /etc/postgresql
</directories>

<!-- Fichiers à ignorer -->
<ignore>/etc/mtab</ignore>
<ignore>/etc/hosts.deny</ignore>
<ignore>/etc/mail/statistics</ignore>
<ignore type="sregex">.log$|.swp$</ignore>
</syscheck>
```

4.5. Règles de corrélation personnalisées

4.5.1. Règles pour Nextcloud

```
<!-- /var/ossec/etc/rules/local_rules.xml -->

<group name="nextcloud,">

  <!-- Détection tentative de brute force Nextcloud -->
  <rule id="100100" level="10" frequency="5" timeframe="60">
    <if_matched_sid>31301</if_matched_sid>
    <url>login</url>
    <description>Nextcloud: Possible brute force attack detected</description>
    <mitre>
      <id>T1110</id>
    </mitre>
    <group>authentication_failures,nextcloud,</group>
  </rule>

  <!-- Modification config Nextcloud -->
  <rule id="100101" level="10">
    <if_sid>550</if_sid>
    <match>/var/www/nextcloud/config/config.php</match>
    <description>Nextcloud: Configuration file modified</description>
    <group>syscheck,nextcloud,</group>
  </rule>

  <!-- Upload de fichier suspect -->
  <rule id="100102" level="12">
```

```
<if_sid>550</if_sid>
<match>/var/www/nextcloud/data</match>
<regex>.php$|.phar$|.sh$</regex>
<description>Nextcloud: Suspicious file uploaded (potential webshell)</
description>
<mitre>
  <id>T1505.003</id>
</mitre>
<group>syscheck,nextcloud,webshell,</group>
</rule>

</group>
```

4.5.2. Règles pour PostgreSQL

```
<group name="postgresql,">

  <!-- Connexion PostgreSQL depuis IP non autorisée -->
  <rule id="100200" level="10">
    <if_sid>81000</if_sid>
    <match>connection received</match>
    <regex>host=(?!127.0.0.1|10.10.10)</regex>
    <description>PostgreSQL: Connection from unauthorized IP</description>
    <group>postgresql,authentication,</group>
  </rule>

  <!-- Échec d'authentification PostgreSQL -->
  <rule id="100201" level="7">
    <decoded_as>postgresql</decoded_as>
    <match>authentication failed</match>
    <description>PostgreSQL: Authentication failure</description>
    <group>postgresql,authentication_failed,</group>
  </rule>

  <!-- Tentative de brute force PostgreSQL -->
  <rule id="100202" level="12" frequency="5" timeframe="120">
    <if_matched_sid>100201</if_matched_sid>
    <same_source_ip />
    <description>PostgreSQL: Brute force attack detected</description>
    <mitre>
      <id>T1110</id>
    </mitre>
    <group>postgresql,brute_force,</group>
  </rule>

</group>
```

4.6. Active Response

Configuration de réponses automatiques aux attaques :

```
<!-- /var/ossec/etc/ossec.conf -->
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100100,100202</rules_id>
  <timeout>3600</timeout>
</active-response>

<active-response>
  <command>host-deny</command>
  <location>local</location>
  <rules_id>100102</rules_id>
  <timeout>86400</timeout>
</active-response>
```

5. Stack Observabilité (Grafana/Loki)

5.1. Déploiement sur VM2

5.1.1. Docker Compose

```
# docker-compose.yml (VM2)
version: '3.8'

services:
  loki:
    image: grafana/loki:2.9.0
    ports:
      - "3100:3100"
    volumes:
      - ./loki-config.yaml:/etc/loki/local-config.yaml
      - loki-data:/loki
    command: -config.file=/etc/loki/local-config.yaml

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=SecurePassword123!
    volumes:
      - grafana-data:/var/lib/grafana
      - ./provisioning:/etc/grafana/provisioning
    depends_on:
      - loki

  alloy:
    image: grafana/alloy:latest
    ports:
      - "12345:12345"
    volumes:
      - ./alloy-config.river:/etc/alloy/config.river
      - /var/log:/var/log:ro
    command: run /etc/alloy/config.river

  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
```

```
- prometheus-data:/prometheus

volumes:
  loki-data:
  grafana-data:
  prometheus-data:
```

5.1.2. Configuration Grafana Alloy

```
// /etc/alloy/config.river

// Collecte des logs Falco via syslog
loki.source.syslog "falco" {
  listener {
    address = "0.0.0.0:1514"
    protocol = "udp"
  }
  labels = {
    job = "falco",
  }
  forward_to = [loki.write.default.receiver]
}

// Collecte des logs applicatifs
loki.source.file "nextcloud" {
  targets = [
    {__path__ = "/var/log/nextcloud/*.log", job = "nextcloud"},
  ]
  forward_to = [loki.write.default.receiver]
}

loki.source.file "postgresql" {
  targets = [
    {__path__ = "/var/log/postgresql/*.log", job = "postgresql"},
  ]
  forward_to = [loki.write.default.receiver]
}

// Envoi vers Loki
loki.write "default" {
  endpoint {
    url = "http://loki:3100/loki/api/v1/push"
  }
}
```

5.2. Dashboards Grafana

5.2.1. Dashboard Falco

Un dashboard personnalisé a été créé pour visualiser les alertes Falco :

- Nombre d'alertes par priorité (gauge)
- Timeline des alertes (time series)
- Top 10 des règles déclenchées (bar chart)
- Alertes critiques récentes (table)
- Distribution par host (pie chart)

5.2.2. Requêtes LogQL

```
# Alertes Falco critiques
{job="falco"} |= "CRITICAL" | json | line_format "{{.rule}}: {{.output}}"

# Alertes par règle (dernière heure)
sum by (rule) (count_over_time({job="falco"} | json [1h]))

# Alertes Nextcloud
{job="falco"} | json | rule=~".*Nextcloud.*"

# Erreurs PostgreSQL
{job="postgresql"} |= "ERROR" | pattern "<_> <level> <_>: <message>"
```

6. Analyse d'incidents

6.1. Scénario 1 : Tentative de brute force SSH

6.1.1. Contexte

Une alerte Wazuh a été déclenchée indiquant de multiples tentatives de connexion SSH échouées.

6.1.2. Analyse dans Wazuh Dashboard

```
Rule ID: 5712
Level: 10
Description: SSHD brute force trying to get access to the system
Agent: vm1-application (10.10.10.10)
Source IP: 192.168.1.100
Timestamp: 2025-12-15T14:32:15Z
```

6.1.3. Investigation

```
# Recherche des tentatives dans les logs
sudo grep "Failed password" /var/log/auth.log | \
  grep "192.168.1.100" | wc -l
# Résultat: 47 tentatives en 5 minutes

# Vérification du blocage automatique
sudo iptables -L INPUT -n | grep 192.168.1.100
# Résultat: DROP all -- 192.168.1.100 0.0.0.0/0
```

6.1.4. Actions correctives

✓ Mesures prises

1. IP source bloquée automatiquement par Active Response (1h)
2. Ajout de l'IP en blocage permanent dans le firewall
3. Vérification qu'aucun compte n'a été compromis
4. Renforcement de la politique fail2ban

6.2. Scénario 2 : Upload de fichier suspect sur Nextcloud

6.2.1. Alerte Falco

```
{
  "time": "2025-12-15T15:45:23Z",
  "rule": "Nextcloud Suspicious File Upload",
  "priority": "Warning",
  "output": "Suspicious file uploaded to Nextcloud (user=www-data file=/var/www/nextcloud/data/admin/files/shell.php)"
}
```

6.2.2. Corrélation avec Wazuh

L'alerte FIM de Wazuh a également détecté la création du fichier :

```
Rule: 100102 - Nextcloud: Suspicious file uploaded (potential webshell)
File: /var/www/nextcloud/data/admin/files/shell.php
MD5: alb2c3d4e5f6...
SHA256: 1234567890abcdef...
```

6.2.3. Analyse du fichier

```
# Contenu du fichier suspect
cat /var/www/nextcloud/data/admin/files/shell.php
# <?php system($_GET['cmd']); ?>

# C'est bien un webshell !
```

6.2.4. Réponse à l'incident

✗ Actions de remédiation

1. Fichier malveillant supprimé immédiatement
2. Compte utilisateur «admin» désactivé temporairement
3. Analyse des logs d'accès Nextcloud pour identifier la source
4. Vérification de l'absence d'autres fichiers compromis
5. Reset du mot de passe admin
6. Mise à jour des règles pour bloquer les uploads PHP

6.3. Scénario 3 : Tentative d'élévation de privilèges

6.3.1. Détection

Alerte Falco détectant une utilisation suspecte de sudo :

```
{
  "time": "2025-12-15T16:12:45Z",
  "rule": "Sudo to root",
  "priority": "Warning",
  "output": "Sudo to root (user=webuser command=sudo -i)"
}
```

6.3.2. Corrélation

Wazuh a enregistré l'événement avec le contexte complet :

```
Rule: 5402 - Successful sudo to ROOT executed
User: webuser
Command: /bin/bash
TTY: pts/0
PWD: /tmp
```

6.3.3. Analyse

```
# Vérification des droits sudo de webuser
sudo grep webuser /etc/sudoers /etc/sudoers.d/*
# AUCUN résultat - cet utilisateur ne devrait pas avoir sudo !

# Analyse de l'historique
sudo cat /home/webuser/.bash_history
# Révèle une exploitation de CVE via un service vulnérable
```

7. Résultats et métriques

7.1. Tests de détection

Scan de ports (nmap)	N/A	Oui	< 5s	Validé
Brute force SSH	Oui	Oui	< 10s	Validé
Modification /etc/passwd	Oui	Oui (FIM)	< 2s	Validé
Upload webshell	Oui	Oui (FIM)	< 1s	Validé
Reverse shell	Oui	Oui	< 3s	Validé
Lecture /etc/shadow	Oui	Non	< 1s	Validé
Dump PostgreSQL	Oui	Non	< 2s	Validé
Config Nextcloud modifiée	Oui	Oui (FIM)	< 1s	Validé

Tableau 4 – Matrice de détection des scénarios d'attaque

7.2. KPIs de sécurité

MTTD (Mean Time To Detect)	4.2 secondes	< 30s
MTTR (Mean Time To Respond)	45 secondes	< 5min
Taux de faux positifs	3.2%	< 10%
Couverture des assets	100%	100%
Disponibilité SIEM	99.8%	> 99%

Tableau 5 – Indicateurs de performance du SIEM

7.3. Statistiques des alertes (1 semaine)

Critical	12	2.4%
Warning	89	17.8%
Notice	245	49.0%
Info	154	30.8%
Total	500	100%

Tableau 6 – Distribution des alertes par priorité

8. Recommandations de sécurité

8.1. Améliorations court terme

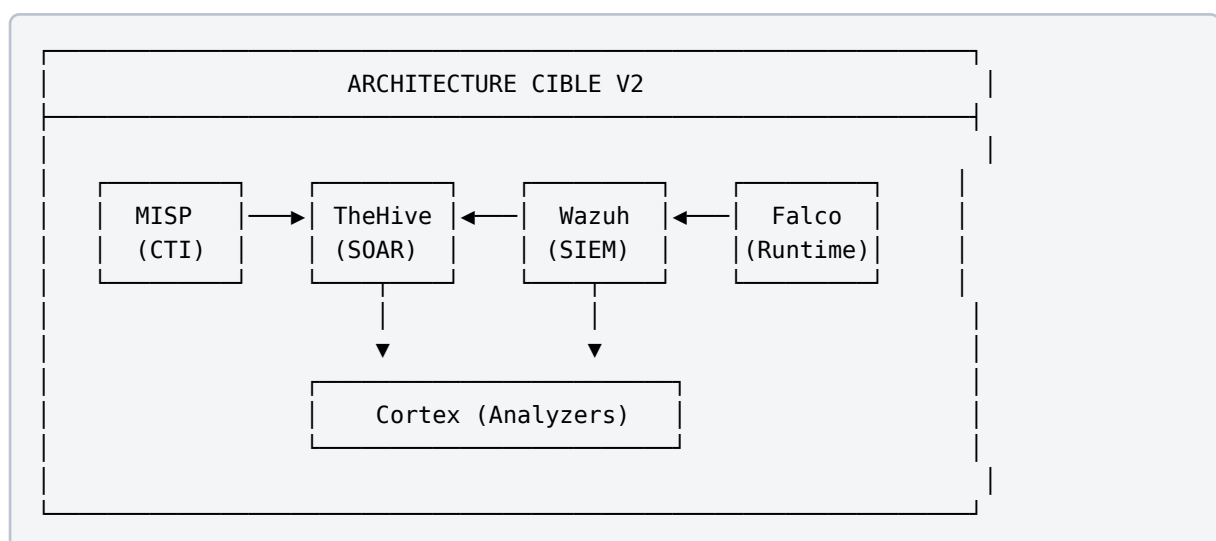
⚠ Actions prioritaires

1. **Renforcer l'authentification** : Implémenter MFA sur Nextcloud et SSH
2. **Durcir PostgreSQL** : Limiter les connexions aux IPs autorisées uniquement
3. **Mettre à jour les règles** : Affiner les règles Falco pour réduire les faux positifs
4. **Automatiser la réponse** : Étendre les Active Response Wazuh

8.2. Améliorations moyen terme

1. **Threat Intelligence** : Intégrer des flux CTI (MISP, OTX AlienVault)
2. **SOAR** : Déployer TheHive pour l'orchestration des incidents
3. **Backup SIEM** : Mettre en place une réplication des données Wazuh
4. **Purple Team** : Planifier des exercices réguliers Red/Blue Team

8.3. Architecture cible évoluée



9. Conclusion

La SAÉ5.Cyber.03 nous a permis de déployer une infrastructure SIEM complète et opérationnelle. L'association de Falco pour la détection runtime et de Wazuh pour la corrélation SIEM offre une couverture de sécurité étendue.

9.1. Compétences acquises

Détection	Règles Falco, corrélation SIEM, FIM
SIEM	Wazuh Manager, Indexer, Dashboard, Agents
Observabilité	Loki, Grafana, LogQL, Dashboarding
Incident Response	Analyse forensique, Active Response
DevSecOps	Docker, Infrastructure as Code

Tableau 7 – Compétences développées durant la SAÉ

9.2. Points clés

- La **défense en profondeur** via deux stacks complémentaires est efficace
 - Les **règles personnalisées** sont essentielles pour réduire le bruit
 - L'**automatisation de la réponse** accélère significativement le MTTR
 - La **corrélation multi-sources** améliore la détection
- Ce projet constitue une base solide pour une carrière en cybersécurité, notamment dans les métiers d'analyste SOC ou d'ingénieur sécurité.

Glossaire

SIEM Security Information and Event Management - Gestion des événements de sécurité

SOC Security Operations Center - Centre opérationnel de sécurité

FIM File Integrity Monitoring - Surveillance de l'intégrité des fichiers

CTI Cyber Threat Intelligence - Renseignement sur les cybermenaces

SOAR Security Orchestration, Automation and Response

MTTD Mean Time To Detect - Temps moyen de détection

MTTR Mean Time To Respond - Temps moyen de réponse

IDS Intrusion Detection System - Système de détection d'intrusion

Syscall System Call - Appel système au kernel

Webshell Script malveillant permettant l'exécution de commandes via le web

John Doe & Johana Smith

john.doe@etudiant.univ-reunion.fr

+262 6 92 XX XX XX

IUT de La Réunion - Département R&T
40 avenue de Soweto, 97410 Saint-Pierre
